

Bonded Authority Tokens

Revoke an agent's authority instantly — without rotating a single key.

Most credentials put authority *in* the token, so revoking one means rotating a key or maintaining a denylist. Bonded tokens put authority in a live, tamper-evident register instead. Mint two tokens together — a **grant** and a **revocation handle** — and you can kill the grant mid-action by flipping the register. The key never changes.

How it works

- **Minted as a pair.** A primary (the grant) and a mirror (the handle) are minted together, each signed, each referencing the other.
- **State lives outside the token.** Authority state — active, suspended, revoked — sits in an append-only, hash-chained ledger, not in the token bytes.
- **Checked live, every request.** A gate handed either token resolves the pair's *current* state from the ledger before allowing an action — so authority is checked against now, not an inherited setup.
- **Revoke without rotation.** Present the mirror, flip the pair to REVOKED, and the primary is dead instantly. No key rotation, no redeploy, no denylist to propagate.

State register

```
ACTIVE_VALIDATED ■■present mirror■■■ REVOKED (terminal)
■
■■■ SUSPENDED / ACTIVE_PENDING / EXPIRED
append-only · hash-chained · one byte-flip breaks verify_chain()
```

Three ways teams use it

Pattern	What it gives you
Action + live monitor	The grant executes; a monitor holds the handle and can halt it mid-action the instant it misbehaves.
Two-party commit	Both halves must move together — built-in dual-control / four-eyes for high-blast-radius operations.
Long-lived grant, live kill	Issue durable authority but keep an instant off-switch that never touches the underlying key.

Why it matters for agentic AI

Agent authority has to be narrow, observable, and revocable *mid-task*. A bearer token can't do that — a drifting or compromised agent keeps acting until you rotate its key. Bonded tokens give a second party standing to stop it atomically, and every transition is signed into a tamper-evident chain an auditor can verify. It pairs directly with Axiom's guest-key delegation (scoped, expiring grants) and signed audit ledgers.

Status

Implemented in `axiom_event_token/bonded_pair.py` with a CLI (`axiom_bonded_pair_cli.py`), a runnable demo, and an integration test suite. Patent disclosure: ORVL-001.

Orivael / Axiom Framework · Confidential.